Universidade POSITIVO

Desenvolvimento de Aplicativos Móveis

Aula 16 — Inserindo Vídeos em Aplicativos

Reproduzindo vídeo local e remoto com expo-video: player, controles e tela cheia

Prof.: MSc. Alex Junior Nunes
pfsalexnunes@gmail.com

CONCEITO

Por Que Vídeo no App?

Vídeo virou parte padrão da experiência de app: onboarding, tutoriais, conteúdo de curso, redes sociais. Hoje aprendemos a peça que faltava pra colocar isso na tela.

ONBOARDING

Vídeo curto explicando como o app funciona na primeira abertura.

CURSOS / AULAS

Conteúdo em vídeo dentro do próprio app, como um player de streaming.

REELS / FEED

Vídeos curtos em loop, com play automático e som desligado.

Instalando o expo-video

```
npx expo install expo-video
```

expo-video é a biblioteca oficial e atual pra vídeo no Expo — substitui o antigo componente Video do expo-av, que está descontinuado. Funciona igual em Android, iOS e Web.

Depois de instalar, reinicie o servidor do Expo (Ctrl+C e npx expo start de novo) pra biblioteca nativa ser carregada — igual fizemos com o Drawer.

As Duas Peças do expo-video

`useVideoPlayer(fonte)`

O "cérebro": controla o vídeo. Guarda a fonte, e dá acesso a play(), pause(), métodos e propriedades como loop e muted.

`<VideoView player={...} />`

A "tela": mostra o vídeo desse player. É um componente comum, com style e props — não controla nada sozinho.

useVideoPlayer é um Hook — a mesma ideia do useSelector do Redux: uma função especial chamada no topo do componente, que devolve algo pra você usar (ali, dados da store; aqui, o player).

Vídeo Local, com require()

```
import { useVideoPlayer, VideoView }
  from 'expo-video';

const fonte = require(
  '../assets/video/apresentacao.mp4'
);

const player = useVideoPlayer(fonte, (p) => {
  p.loop = true;
  p.play();
});
```

O segundo argumento

é uma função de configuração inicial: roda uma vez, recebe o player pronto (p) e ajusta o que precisar antes da tela aparecer.

Vídeos locais entram em assets/, do mesmo jeito que já fazemos com imagens — o Metro empacota o arquivo no app.

Vídeo Remoto, com URL

```
const fonte =
  'https://commondatastorage.googleapis.com/gtv-videos-
bucket/sample/BigBuckBunny.mp4';

const player = useVideoPlayer(fonte);

<VideoView
  style={{ width: '100%', height: 220 }}
  player={player}
/>
```

Praticamente igual ao local

A fonte agora é uma string com a URL, em vez de require(). useVideoPlayer e VideoView não mudam nada.

Se o vídeo demorar pra carregar, o VideoView fica com a tela preta até o buffer inicial chegar — vale mostrar um indicador de carregamento por cima.

Vídeo remoto consome dados do usuário e depende de internet — pra conteúdo fixo do app (tutorial, onboarding), local costuma ser a escolha mais confiável.

Controles Básicos do Player

```
<TouchableOpacity
  onPress={() => {
    player.playing
      ? player.pause()
      : player.play();
  }}
>
  <Text>
    {player.playing ? 'Pausar' : 'Tocar'}
  </Text>
</TouchableOpacity>
```

`player.play() / pause()`	controla a reprodução
`player.loop = true`	repete o vídeo ao terminar
`player.muted = true`	toca sem som
`player.playing`	true enquanto está tocando

Estilizando o VideoView

```
<VideoView
  player={player}
  style={{
    width: '100%',
    height: 220,
    borderRadius: 12,
  }}
  contentFit="contain"
  nativeControls={false}
/>
```

contentFit

"contain" / "cover" / "fill" — igual ao resizeMode de Image

nativeControls

mostra (true) ou esconde (false) os controles nativos do sistema

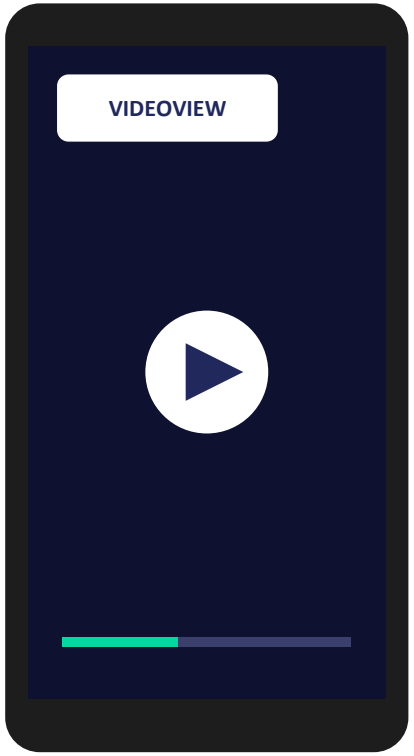
borderRadius

vem do style, um <View> comum por baixo

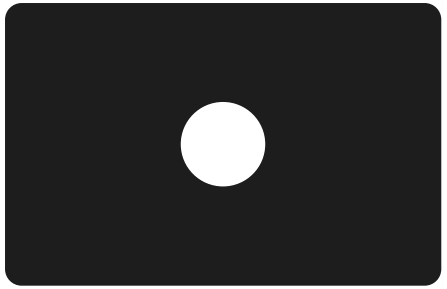
Tela Cheia e Picture-in-Picture

```
<VideoView
  player={player}
  allowsFullscreen
  allowsPictureInPicture
  style={{ width: '100%', height: 220 }}
/>
```

Com nativeControls ligado, o próprio botão de tela cheia já aparece — as props só habilitam a permissão. O sistema cuida do resto.



Player normal



Picture-in-Picture: o vídeo continua tocando numa janelinha por cima de outro app.

Cuidados ao Sair da Tela

Um player de vídeo continua consumindo recursos mesmo se a tela some — vale saber o que o expo-video já resolve sozinho, e o que ainda é responsabilidade sua.

O expo-video já cuida

useVideoPlayer é um Hook — o mesmo mecanismo do Redux que já limpa inscrições automaticamente. Ao desmontar a tela, o player é liberado sozinho, sem código extra.

Continua sendo sua

Pausar o vídeo ao navegar pra outra tela (senão o áudio continua tocando por baixo) e mostrar algum aviso enquanto a internet carrega um vídeo remoto.

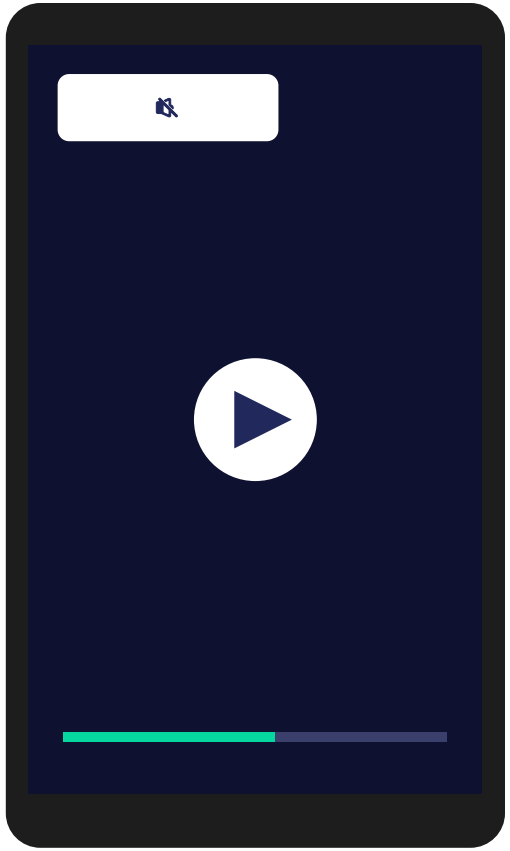
DESAFIO

Desafio da Aula: Player Customizado

Em duplas, montem uma tela com um vídeo local e:

- [ ] VideoView mostrando o vídeo, com contentFit="cover"
- [ ] Botão próprio de Tocar / Pausar (sem nativeControls)
- [ ] Botão para ligar/desligar o som (player.muted)
- [ ] Vídeo em loop desde o início (player.loop = true)

`Pasta: 06-player-customizado/ · Gabarito: 06-player-customizado-solucao/`



Como deve ficar

O Que Levamos de Hoje

- expo-video é a forma atual de tocar vídeo no Expo — local ou remoto
- useVideoPlayer cria e controla o player; VideoView só exibe
- play(), pause(), loop e muted controlam a reprodução por código
- contentFit e nativeControls ajustam a aparência do player
- allowsFullscreen e allowsPictureInPicture habilitam os dois recursos

Próxima aula: 01/10 — Prova Teórica. Revisem navegação (Stack, Tab, Drawer), Redux e vídeo com expo-video.